法が素数の場合のカードベース乗算プロトコル

品川 和雅[1,2,3]

概要：カードベース暗号では、整数剰余環 $\mathbb{Z}_m$ の元を m 枚のカードで表現する整数符号化が知られている。この符号化に対し、加算は追加カードを用いることなく 1 回のパイルシフティングシャッフルで実現できる一方、既存の乗算プロトコルは $m^2 - m$ 枚の追加カードと $3m - 7$ 回のパイルシフティングシャッフルを必要とする。本研究では、まず、既存方式と同じ枚数のカードを用いながら、乗算に必要なパイルシフティングシャッフルの回数を $\lceil \log_2 m \rceil + 1$ 回に削減するプロトコルを提案する。次に、m が素数の場合には、追加カードを $2m - 2$ 枚、パイルシフティングシャッフルを 2 回にまで削減した、より効率的な乗算プロトコルを提案する。特に、後者の 2 回のシャッフルは、途中でカードを公開することなく連続して実行されるため、これらを合成し、1 回のシャッフルとして実行することも可能である。

キーワード：カードベース暗号、整数符号化、乗算プロトコル、パイルシフティングシャッフル

Card-Based Multiplication Protocols for Prime Moduli

Kazumasa Shinagawa[1,2,3]

Abstract: In card-based cryptography, an integer encoding is known that represents an element of the residue ring $\mathbb{Z}_m$ using m cards. Under this encoding, addition can be performed using a single pile-shifting shuffle without any helping cards, whereas the existing multiplication protocol uses $m^2 - m$ helping cards and $3m - 7$ pile-shifting shuffles. In this work, we first propose a multiplication protocol that reduces the number of pile-shifting shuffles to $\lceil \log_2 m \rceil + 1$ while using the same number of cards as the existing protocol. Furthermore, when m is prime, we propose a more efficient multiplication protocol that requires only $2m - 2$ helping cards and two pile-shifting shuffles. In particular, since these two shuffles are performed consecutively without revealing any cards in between, they can be composed and executed as a single shuffle.

Keywords: card-based cryptography, integer encoding, multiplication protocols, pile-shifting shuffles

1. はじめに

1.1 背景

カードベース暗号 [1–3] とは、物理的なカードを用いて秘密計算やゼロ知識証明などの暗号プロトコルを実現する研究分野である。秘密計算に関しては、2 枚のカード ♣♡ の並びによってビットを表現し、論理回路を評価するプロトコルが古くから研究されてきた。特に、Mizuki と Sone [4] が提案した AND/XOR/コピープロトコルは、いずれもランダム二等分割カットを 1 回だけ用いる簡潔なものである。これらのプロトコルを組み合わせることで、任意の論理回路に対する秘密計算プロトコルが得られる。

論理回路の研究と並行して、算術回路の研究も進められている。算術回路のプロトコルでは、整数剰余環

$$\mathbb{Z}_m := \mathbb{Z}/m\mathbb{Z} = \{0, 1, \ldots, m-1\}$$

の元を m 枚のカードで表現する整数符号化を用いる。整数符号化 $\mathsf{E}_m(x)$ は、左端を 0 番目とし、x 番目に ♡ を置き、それ以外の位置に ♣ を置く符号化である。

Ruangwises と Itoh [6] は、$\mathsf{E}_m(x)$ と $\mathsf{E}_m(y)$ から $\mathsf{E}_m(x+y)$ を計算する加算プロトコルを構成した。このプロトコルは追加カードを必要とせず、パイルシフティングシャッフル

[1] 筑波大学 University of Tsukuba
[2] 九州大学 Kyushu University
[3] 産業技術総合研究所 National Institute of Advanced Industrial Science and Technology

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

方式	追加カード枚数	シャッフル回数	シャッフルの種類	備考
Ruangwises [5]	m^2-m	$3m-7$	パイルシフティングシャッフル	任意 m
提案プロトコル（3 節）	m^2-m	$\lceil \log_2 m \rceil + 1$	パイルシフティングシャッフル	任意 m
提案プロトコル（4.1 節）	$2m-2$	2	パイルシフティングシャッフル	素数 m
提案プロトコル（4.4 節）	$2m-2$	1	一様閉シャッフル	素数 m

表 1 整数符号化 $E_m(\cdot)$ に対する乗算プロトコルの効率性の比較

を 1 回だけ用いる。また、コピープロトコルもパイルシフティングシャッフルを 1 回だけ用いて構成できる [5]。

加算プロトコルが効率的に実現できるのに対し、乗算プロトコル [5] の追加カード枚数は m^2-m 枚、シャッフル回数は $3m-7$ 回であり、加算プロトコルよりも大きなコストが必要であった。プロトコルの基本方針は、コピープロトコルと加算プロトコルを繰り返し用いて

$$(\mathsf{E}_m(0), \mathsf{E}_m(x), \mathsf{E}_m(2x), \mathsf{E}_m(3x), \ldots, \mathsf{E}_m((m-1)x))$$

を作成し、その中から y 番目の $\mathsf{E}_m(yx)$ を選択するというものである。追加カード枚数が $O(m^2)$ 枚になるのは、各 m 枚必要な整数符号化を m 個用意することが理由である。

1.2 本研究の貢献

本研究では、$\mathsf{E}_m(x)$ と $\mathsf{E}_m(y)$ から $\mathsf{E}_m(xy)$ を計算する乗算プロトコルを 3 つ提案する。各方式の性能を表 1 に示す。

1 つ目のプロトコルは、既存プロトコルと同じカード枚数を用いて、パイルシフティングシャッフルの回数を $\lceil \log_2 m \rceil + 1$ 回に削減した方式である。これはバイナリ法を自然に適用することで実現できる。

2 つ目のプロトコルは、m が素数の場合に限り適用可能な方式であり、追加カード枚数 $2m-2$ 枚と 2 回のパイルシフティングシャッフルを用いて乗算を実現する。1 つ目のプロトコルと比べて、追加カード枚数を $O(m^2)$ 枚から $O(m)$ 枚へ削減し、シャッフル回数を $O(\log m)$ 回から 2 回へ削減した。したがって、m の適用範囲が素数に限定されるものの、いずれの指標に関しても既存方式および 1 つ目のプロトコルより優れている。

3 つ目のプロトコルは、2 つ目のプロトコルのシャッフル回数をさらに削減し、パイルシフティングシャッフル 1 回を用いて乗算を実現する。その基本アイディアは、2 つ目のプロトコルはシャッフルとシャッフルの間に途中でカードをめくらないため、シャッフル 1 回に合成できるというものである。なお、合成後のシャッフルは一様閉シャッフルである。3 つ目のプロトコルも、2 つ目のプロトコルと同様、m が素数の場合に限り適用可能な方式である。

2. 既存プロトコル

2.1 パイルシフティングシャッフル (PShift)

パイルシフティングシャッフル (Pile-Shifting Shuffle: PShift) [8] とは、同じ枚数からなるカード束たちに対して、ランダムな巡回シフトを適用するシャッフル操作である。特に、カード束 2 個の場合は**ランダム二等分割カット** (Random Bisection Cut: RBC) [4] と呼ばれる。

2.2 Ruangwises–Itoh の加算プロトコル

Ruangwises と Itoh [6] は 2 つの整数符号化 $\mathsf{E}_m(x) = (X_0, X_1, \ldots, X_{m-1})$ と $\mathsf{E}_m(y) = (Y_0, Y_1, \ldots, Y_{m-1})$ を入力として受け取り、$\mathsf{E}_m(x+y)$ を出力する加算プロトコルを構成した。1 回の PShift を用い、追加カードは不要である。

(1) $\mathsf{E}_m(y)$ の $(Y_1, \ldots, Y_{m-1})$ を逆順に並べ替える。

$$\underbrace{(Y_0, Y_1, Y_2, \ldots, Y_{m-1})}_{\mathsf{E}_m(y)} \to \underbrace{(Y_0, Y_{m-1}, \ldots, Y_2, Y_1)}_{\mathsf{E}_m(-y)}$$

並べ替えた後のカード列を $(Z_0, Z_1, \ldots, Z_{m-1})$ とおくと、これは $\mathsf{E}_m(-y)$ を表す。

(2) $\mathsf{E}_m(x)$ の下に $\mathsf{E}_m(-y)$ を並べた以下の行列を作成する。

$$\begin{pmatrix} X_0 & X_1 & X_2 & \cdots & X_{m-1} \\ Z_0 & Z_1 & Z_2 & \cdots & Z_{m-1} \end{pmatrix}$$

(3) 各列を一つの束とし、PShift を適用する。その結果、ある一様乱数 $r \in \mathbb{Z}_m$ について、上段は $\mathsf{E}_m(x+r)$ となり、下段は $\mathsf{E}_m(-y+r)$ となる。

(4) 下段のカードをめくり、$\epsilon := -y + r$ を公開する。

(5) 上段のカード列に ϵ 回の左シフトを適用し、$\mathsf{E}_m(x + r - \epsilon) = \mathsf{E}_m(x+y)$ を得る。これを出力する。

2.3 Ruangwises のコピープロトコル

Ruangwises [5] は $\mathsf{E}_m(x) = (X_0, X_1, \ldots, X_{m-1})$ を入力として受け取り、2 個の $\mathsf{E}_m(x)$ を出力するコピープロトコルを構成した。1 回の PShift と $3m$ 枚のカードを用いる。

(1) $\mathsf{E}_m(x)$ の $(X_1, \ldots, X_{m-1})$ を逆順に並べ替える。

$$\underbrace{(X_0, X_1, X_2, \ldots, X_{m-1})}_{\mathsf{E}_m(x)} \to \underbrace{(X_0, X_{m-1}, \ldots, X_2, X_1)}_{\mathsf{E}_m(-x)}$$

並べ替えた後のカード列を $(Z_0, Z_1, \ldots, Z_{m-1})$ とおくと、これは $\mathsf{E}_m(-x)$ を表す。

(2) 以下の行列を作成する。

$$\begin{pmatrix} Z_0 & Z_1 & Z_2 & \cdots & Z_{m-1} \\ \heartsuit & \clubsuit & \clubsuit & \cdots & \clubsuit \\ \heartsuit & \clubsuit & \clubsuit & \cdots & \clubsuit \end{pmatrix}$$

このとき ♡ および ♣ はすべて裏向きに置いている。2 段目と 3 段目のカード列を $(W_0, W_1, \ldots, W_{m-1})$ とお

© 2026 Information Processing Society of Japan

くと、これは $\mathsf{E}_m(0)$ を表す。

(3) 各列を束として PShift を適用する。その結果、ある一様乱数 $r \in \mathbb{Z}_m$ について、1 段目は $\mathsf{E}_m(-x+r)$ となり、2 段目と 3 段目は $\mathsf{E}_m(r)$ となる。

(4) 上段のカードをめくり、$\epsilon := -x + r$ を公開する。

(5) 2 段目と 3 段目のカード列に ϵ 回の左シフトを適用し、$\mathsf{E}_m(r-\epsilon) = \mathsf{E}_m(x)$ を 2 個得る。これを出力する。

2.4 Ruangwises の乗算プロトコル

Ruangwises [5] は $\mathsf{E}_m(x) = (X_0, X_1, \ldots, X_{m-1})$ と $\mathsf{E}_m(y) = (Y_0, Y_1, \ldots, Y_{m-1})$ を入力として受け取り、$\mathsf{E}_m(xy)$ を出力する乗算プロトコルを構成した。$3m-7$ 回の PShift を用い、追加カードは $m^2 - m$ 枚である。

(1) $A_1 := \mathsf{E}_m(x)$ とする。$m \geq 4$ なら手順 (2) に進み、$m = 3$ なら手順 (3) に進む。

(2) $i = 1, 2, \ldots, m-3$ について以下を実行する。

(a) 現在までに $A_1, \ldots, A_i$ が得られている。ただし A_j（$1 \leq j \leq i$）は $\mathsf{E}_m(jx)$ である。

(b) コピープロトコルを適用し、A_1 を 2 個に増やす。

(c) コピープロトコルを適用し、A_i を 2 個に増やす。

(d) A_1 と A_i に加算プロトコルを適用し、$\mathsf{E}_m((i+1)x)$ を得る。これを A_{i+1} とする。

(3) 手順 (2) によって $A_1, \ldots, A_{m-2}$ が得られている。コピープロトコルを適用し、A_1 を 2 個に増やす。

(4) $A_1 = \mathsf{E}_m(x) = (X_0, X_1, \ldots, X_{m-1})$ に対して、右側 $m-1$ 枚を逆順にし、$\mathsf{E}_m(-x) =: A_{m-1}$ を作る。

(5) 以下の行列を作り、各列を束として PShift を適用する。

$$\begin{pmatrix} Y_0 & Y_1 & Y_2 & \cdots & Y_{m-1} \\ \mathsf{E}_m(0) & A_1 & A_2 & \cdots & A_{m-1} \end{pmatrix}$$

(6) 1 行目のカードをめくり、♡ の下の m 枚を出力する。

$$\begin{pmatrix} \clubsuit & \cdots & \heartsuit & \cdots & \clubsuit \\ * & \cdots & \mathsf{E}_m(xy) & \cdots & * \end{pmatrix}$$

3. 任意法の乗算プロトコル

本節では $\lceil \log_2 m \rceil$ 回の乗算プロトコルを提案する。この方式は、正多角形カードに対する乗算プロトコル [7] の整数符号化版であり、バイナリ法を用いている。

3.1 定数倍プロトコル

定数倍プロトコルは、整数符号化 $\mathsf{E}_m(x)$ と、$m-1$ 個の $\mathsf{E}_m(0)$ が与えられているとき、

$$S^* = (\mathsf{E}_m(0), \mathsf{E}_m(x), \mathsf{E}_m(2x), \ldots, \mathsf{E}_m((m-1)x))$$

を出力するプロトコルである。

ここである $a_0, a_1, \ldots, a_{m-1} \in \mathbb{Z}_m$ に対して

$$S = (\mathsf{E}_m(a_0 x), \mathsf{E}_m(a_1 x), \mathsf{E}_m(a_2 x), \ldots, \mathsf{E}_m(a_{m-1} x))$$

とする。カード列 S が i 完全であるとは、

$$(a_1, a_2, \ldots, a_i) = (1, 2, \ldots, i)$$

を満たすことと定める。（a_0 の値は気にしていないことに注意する。）また S の j 番目の容量を $j - a_j$ とする。

以降、一番左を 0 番目とし、i 番目の整数符号化を X_i で参照することにする。すなわち、X_i はその時点での「左から i 番目の整数符号化」を指し示す変数である。

(1) 以下のカード列を作る。

$$(\mathsf{E}_m(0), \mathsf{E}_m(x), \mathsf{E}_m(0), \ldots, \mathsf{E}_m(0))$$

このカード列は 1 完全である。また、$i \geq 2$ において i 番目の容量は i である。

(2) 以下の手順を S^* を得られるまで繰り返す。

(a) $i^* \in \mathbb{Z}_m$ を、現在のカード列 S について、以下の条件を満たす最大の整数 i とする。

- S は i 完全である。
- S の $m-1$ 番目の容量は t 以上である。

(b) 0 番目と i^* 番目の整数符号化（X_0 と X_{i^*}）を交換する。この交換により、現在は $X_0 = \mathsf{E}_m(i^* x), X_{i^*} = \mathsf{E}_m(0)$ が成り立つ。（i 完全の定義で a_0 を除外していたのは、この状況のためである。）

(c) $T \subseteq \mathbb{Z}_m$ を、現在のカード列 S について、以下の条件を満たす最大 $t \in \mathbb{Z}_m$ を集めた集合とする。

- S は t 完全ではない。
- S の t 番目の容量は i^* 以上である。

なお、S の i^* 番目の容量は i^* であるため、$i^* \in T$ は必ず成り立つことに注意する。

(d) Ruangwises の加算プロトコルを用いて、X_0 の値を X_t（$t \in T$）に同時に足す。この結果、各 X_t（$t \in T$）の符号化に $i^* x$ が足され、X_0 には表向きの m 枚のカードが置かれている状態になる。これは 1 回の PShift によって実現できる。

(e) X_0 の m 枚のカードを伏せて $X_0 = \mathsf{E}_m(0)$ とする。

このプロトコルのシャッフル回数は高々 $\lceil \log_2 m \rceil$ 回であり、シャッフルはすべて PShift である。

3.2 乗算プロトコル

$\mathsf{E}_m(x)$ と $\mathsf{E}_m(y) = (Y_0, Y_1, \ldots, Y_{m-1})$ を入力として受け取り、$\mathsf{E}_m(xy)$ を出力する乗算プロトコルを構成する。

(1) $\mathsf{E}_m(x)$ と $m(m-1)$ 枚の追加カード（内訳は $m-1$ 枚の ♡ と $(m-1)^2$ 枚の ♣）を用いて、以下を作成する。

$$(\mathsf{E}_m(x), \underbrace{\mathsf{E}_m(0), \mathsf{E}_m(0), \ldots, \mathsf{E}_m(0)}_{m-1 \text{ 個}})$$

(2) 前節の定数倍プロトコルを用いてカード列 S^* を作る。

$$S^* = (\mathsf{E}_m(0), \mathsf{E}_m(x), \mathsf{E}_m(2x), \ldots, \mathsf{E}_m((m-1)x))$$

© 2026 Information Processing Society of Japan

(3) 以下の行列を作り、PShift を適用する。

$$\begin{pmatrix} Y_0 & Y_1 & Y_2 & \cdots & Y_{m-1} \\ \mathsf{E}_m(0) & \mathsf{E}_m(x) & \mathsf{E}_m(2x) & \cdots & \mathsf{E}_m((m-1)x) \end{pmatrix}$$

(4) 1 行目のカードをめくり、♡ の下の m 枚を出力する。

$$\begin{pmatrix} \clubsuit & \cdots & \heartsuit & \cdots & \clubsuit \\ * & \cdots & \mathsf{E}_m(xy) & \cdots & * \end{pmatrix}$$

このプロトコルのシャッフル回数は高々 $\lceil \log_2 m \rceil + 1$ 回であり、シャッフルはすべて PShift である。

4. 素数法の乗算プロトコル

これまで整数剰余環の法を m と表記していたが、以降では素数の場合のみ扱うため、代わりに p を用いる。

4.1 乗算プロトコル

$p = 5$ の場合の乗算プロトコルの手順を以下に示す。一般の素数 p の場合も同様の手順で実行できる。

以下の整数符号化が与えられているとする。

- $\mathsf{E}_5(x) = (X_0, X_1, X_2, X_3, X_4)$
- $\mathsf{E}_5(y) = (Y_0, Y_1, Y_2, Y_3, Y_4)$

X_i, Y_i はそれぞれ $\mathsf{E}_5(x), \mathsf{E}_5(y)$ の i 番目のカードを表す。

この並び順は乗法とは相性が悪いため、乗法と相性の良い新しい並び順を用いることにする。乗法群 $\mathbb{Z}_5^\times$ の生成元 g を一つ固定する。例えば $g = 2$ と固定する。このとき

- $\mathsf{E}_5^\times(x) = (X_{g^0}, X_{g^1}, X_{g^2}, X_{g^3}) = (X_1, X_2, X_4, X_3)$
- $\mathsf{E}_5^\times(y) = (Y_{g^0}, Y_{g^1}, Y_{g^2}, Y_{g^3}) = (Y_1, Y_2, Y_4, Y_3)$

という符号化を用いる。これは $x = 0$ ならすべて ♣ であり、$x \neq 0$ なら $x = g^i$ のとき i 番目が ♡ の符号化である。

乗算プロトコルの手順は以下の通りである。

(1) $\mathsf{E}_5(x)$ を並べ替え、$(X_0, \mathsf{E}_5^\times(x))$ を作る。

$$\underbrace{(X_0, X_1, X_2, X_3, X_4)}_{\mathsf{E}_5(x)} \to (X_0, \underbrace{X_1, X_2, X_4, X_3}_{\mathsf{E}_5^\times(x)})$$

同様に、$\mathsf{E}_5(y)$ から $(Y_0, \mathsf{E}_5^\times(y))$ を作る。

$$\underbrace{(Y_0, Y_1, Y_2, Y_3, Y_4)}_{\mathsf{E}_5(y)} \to (Y_0, \underbrace{Y_1, Y_2, Y_4, Y_3}_{\mathsf{E}_5^\times(y)})$$

(2) $\mathsf{E}_5^\times(y)$ の右側 3 枚を逆順に並べ替え、$\mathsf{E}_5^\times(y^{-1})$ を作る。

$$\underbrace{(Y_1, Y_2, Y_4, Y_3)}_{\mathsf{E}_5^\times(y)} \to \underbrace{(Y_1, Y_3, Y_4, Y_2)}_{\mathsf{E}_5^\times(y^{-1})}$$

(3) $(X_0, \mathsf{E}_5^\times(x))$ と $\mathsf{E}_5^\times(y^{-1})$ と 3 枚の ♣ から以下を作る。

$$\begin{pmatrix} \mathsf{E}_5^\times(x) \\ X_0\clubsuit\clubsuit\clubsuit \\ \mathsf{E}_5^\times(y^{-1}) \end{pmatrix} = \begin{pmatrix} X_1 & X_2 & X_4 & X_3 \\ X_0 & \clubsuit & \clubsuit & \clubsuit \\ Y_1 & Y_3 & Y_4 & Y_2 \end{pmatrix}$$

このとき 3 枚の ♣ は裏向きに置いている。

(4) 各列 3 枚を 1 つの束とし、PShift を適用する。その結果の行列を以下のように表す。

$$\begin{pmatrix} A_0 & A_1 & A_2 & A_3 \\ B_0 & B_1 & B_2 & B_3 \\ C_0 & C_1 & C_2 & C_3 \end{pmatrix}$$

このとき各行は以下のように表せる。

- $(A_0, A_1, A_2, A_3) = \mathsf{E}_5^\times(x \cdot r)$
- $(B_0, B_1, B_2, B_3) = \mathsf{E}_5^\times(\delta(x) \cdot r)$
- $(C_0, C_1, C_2, C_3) = \mathsf{E}_5^\times(y^{-1} \cdot r)$

ここで $r \in \mathbb{Z}_m^\times$ は PShift によって生成された一様乱数である。また $\delta(x)$ は $x = 0$ のとき 1 で、それ以外のとき 0 を返す関数である。

(5) $\mathsf{E}_5^\times(y^{-1} \cdot r)$ の右側 3 枚を逆順にし、$\mathsf{E}_5^\times(y \cdot r^{-1})$ を作る。

$$\underbrace{(C_0, C_1, C_2, C_3)}_{\mathsf{E}_5^\times(y^{-1} \cdot r)} \to \underbrace{(C_0, C_3, C_2, C_1)}_{\mathsf{E}_5^\times(y \cdot r^{-1})}$$

これを $(D_0, D_1, D_2, D_3) := (C_0, C_3, C_2, C_1)$ とおく。

(6) 以下の行列を作成する。

$$\begin{pmatrix} \mathsf{E}_5^\times(x \cdot r) & Y_0 & \mathsf{E}_5^\times(y \cdot r^{-1}) \\ \mathsf{E}_5^\times(\delta(x) \cdot r) & \heartsuit & \clubsuit\clubsuit\clubsuit\clubsuit \end{pmatrix} = \begin{pmatrix} A_0A_1A_2A_3 & Y_0 & D_0D_1D_2D_3 \\ B_0B_1B_2B_3 & \heartsuit & \clubsuit\clubsuit\clubsuit\clubsuit \end{pmatrix}$$

このとき ♡ および ♣ はすべて裏向きに置いている。

(7) 各行を 1 つの束とし、PShift（束が 2 個なので RBC）を適用する。その結果の行列を以下のように表す。

$$\begin{pmatrix} \alpha_0\alpha_1\alpha_2\alpha_3 & \gamma_0 & \epsilon_0\epsilon_1\epsilon_2\epsilon_3 \\ \beta_0\beta_1\beta_2\beta_3 & \zeta_0 & \eta_0\eta_1\eta_2\eta_3 \end{pmatrix}$$

(8) 左側 4 列（$\alpha_0\alpha_1\alpha_2\alpha_3$ と $\beta_0\beta_1\beta_2\beta_3$）をすべてめくる。めくった 8 枚のうち 1 枚だけ ♡ である。一般性を失わずに α の方に ♡ が現れたとする。（もし β の方に ♡ が現れたら、行全体を交換すればよい。）♡ の位置に応じて ϵ を以下のようにシフトする。

$\alpha_0\alpha_1\alpha_2\alpha_3$	シフト量	シフト後の ϵ
♡♣♣♣	0	$\epsilon_0\epsilon_1\epsilon_2\epsilon_3$
♣♡♣♣	1	$\epsilon_3\epsilon_0\epsilon_1\epsilon_2$
♣♣♡♣	2	$\epsilon_2\epsilon_3\epsilon_0\epsilon_1$
♣♣♣♡	3	$\epsilon_1\epsilon_2\epsilon_3\epsilon_0$

シフト後の ϵ を $(\theta_0, \theta_1, \theta_2, \theta_3)$ と表す。

(9) $(\gamma_0, \theta_0, \theta_1, \theta_2, \theta_3)$ に対して手順 (1) の逆置換を適用し、これを出力する。

$$(\gamma_0, \theta_0, \theta_1, \theta_2, \theta_3) \to \underbrace{(\gamma_0, \theta_0, \theta_1, \theta_3, \theta_2)}_{\mathsf{E}_5(xy)}$$

提案プロトコルは 2 回の PShift（内訳：1 回の PShift と

© 2026 Information Processing Society of Japan

1 回の RBC）と $2p-2$ 枚の追加カードを用いる。追加カードの内訳は、手順 (3) で用いる ♣ カード $p-2$ 枚と、手順 (6) で用いる ♡ カード 1 枚および ♣ カード $p-1$ 枚である。4.4 節で 1 回のシャッフルに合成した場合について論じる。

4.2 正当性

$a \in \mathbb{Z}_p^\times$ に対して、$a = g^i$ となる i を $\log_g a$ と表記する。

まず $x \neq 0$ の場合を考える。ひとまず $y \neq 0$ と仮定し、

$$i = \log_g x,\ j = \log_g y,\ k = \log_g r$$

とおく。ただし r は手順 (4) の PShift の乱数である。以降、i, j, k に関する計算はすべて $\bmod(p-1)$ で考える。

手順 (4) において、$A_{i+k} = \heartsuit$ および $C_{-j+k} = \heartsuit$ である。手順 (5) において $D_{j-k} = \heartsuit$ である。手順 (8) において α（♡ を含む方）は $\mathsf{E}_p^\times(x \cdot r)$ であるから、$\alpha_{i+k} = \heartsuit$ であり、シフト量は $i+k$ である。すなわち θ は $\mathsf{E}_p^\times(xy)$ であり、手順 (9) は $\mathsf{E}_p(xy)$ を正しく出力する。また、$y = 0$ のとき $\gamma_0 = \heartsuit$ であり、この場合も $\mathsf{E}_p(0)$ を正しく出力する。

次に $x = 0$ の場合を考える。このとき手順 (8) において α は $\mathsf{E}_p^\times(\delta(x) \cdot r)$ であり、$(\gamma_0, \epsilon_0\epsilon_1 \cdots \epsilon_{p-2}) = (\heartsuit, \clubsuit\clubsuit\cdots\clubsuit)$ である。よってシフト量に関わらず、$\theta = \clubsuit\clubsuit\cdots\clubsuit$ であり、手順 (9) で $\mathsf{E}_p(0)$ を正しく出力する。

4.3 安全性

プロトコル中にめくるカードは手順 (8) の $\alpha = \alpha_0\alpha_1\cdots\alpha_{p-2}$ と $\beta = \beta_0\beta_1\cdots\beta_{p-2}$ のみである。これらの中に含まれる ♡ は 1 枚であり、残りはすべて ♣ である。PShift と RBC の効果により、♡ の現れる位置は一様ランダムである。したがって、提案プロトコルは安全である。

4.4 シャッフル 1 回のプロトコル

手順 (4) と手順 (7) のシャッフルの間にカードをめくらないため、シャッフルを単純に合成することで、シャッフル 1 回のプロトコルが得られる。本節では、単純な合成よりも見通しのよいシャッフル 1 回のプロトコルを提案する。

用いるシャッフルは、以下のシャッフルである。

$$\begin{pmatrix} a_{0,0}a_{0,1}\cdots a_{0,p-2} & b_0 & c_{0,0}c_{0,1}\cdots c_{0,p-2} \\ a_{1,0}a_{1,1}\cdots a_{1,p-2} & b_1 & c_{1,0}c_{1,1}\cdots c_{1,p-2} \end{pmatrix}$$
$$\to \begin{pmatrix} a_{r,s}a_{r,s+1}\cdots a_{r,s-1} & b_r & c_{r,s}c_{r,s+1}\cdots c_{r,s-1} \\ a_{\overline{r},s}a_{\overline{r},s+1}\cdots a_{\overline{r},s-1} & b_{\overline{r}} & c_{\overline{r},s}c_{\overline{r},s+1}\cdots c_{\overline{r},s-1} \end{pmatrix}$$

ここで $r \in \{0,1\}$ および $s \in \{0,1,\ldots,p-2\}$ は一様乱数であり、右側の添え字は $\bmod(p-1)$ を取る。また、$\overline{r} := 1-r$ である。すなわち、中央の b を除き、左右の $p-1$ 枚を同じ乱数 s でランダムシフトし、1 行目と 2 行目を入れ替えるシャッフルである。これは一様閉シャッフルである。

このシャッフルを用いるプロトコルの手順は以下である。

(1) 4.1 節の手順 (1) と (2) を実行する。

(2) 以下の行列を作成し、上述のシャッフルを適用する。

$$\begin{pmatrix} \mathsf{E}_5^\times(x) & Y_0 & \mathsf{E}_5^\times(y) \\ X_0\clubsuit\clubsuit\clubsuit & \heartsuit & \clubsuit\clubsuit\clubsuit\clubsuit \end{pmatrix}$$

その結果の行列を以下のように表す。

$$\begin{pmatrix} \alpha_0\alpha_1\alpha_2\alpha_3 & \gamma_0 & \epsilon_0\epsilon_3\epsilon_2\epsilon_1 \\ \beta_0\beta_1\beta_2\beta_3 & \zeta_0 & \eta_0\eta_3\eta_2\eta_1 \end{pmatrix}$$

(3) ϵ と η のそれぞれに対して 4.1 節の手順 (2) と同様の並べ替えを行う。その結果、以下の行列を得る。

$$\begin{pmatrix} \alpha_0\alpha_1\alpha_2\alpha_3 & \gamma_0 & \epsilon_0\epsilon_1\epsilon_2\epsilon_3 \\ \beta_0\beta_1\beta_2\beta_3 & \zeta_0 & \eta_0\eta_1\eta_2\eta_3 \end{pmatrix}$$

(4) 4.1 節の手順 (8) と (9) を実行する。

ここで 4.1 節のプロトコルのシャッフルを単純に合成した場合との違いは、上記の手順 (2) で置いた ♣♣♣♣ にランダムシフトを適用するか否かである。このプロトコルの正当性と安全性も 4.1 節のプロトコルと同様に成り立つ。

謝辞　本研究は JSPS 科研費 JP26K21162、JP21K17702 と JST CREST JPMJCR22M1 と JST 経済安全保障重要技術育成プログラム JPMJKP24U2 の支援を受けている。

参考文献

[1] C. Crépeau and J. Kilian. Discreet solitary games. In D. R. Stinson ed., *Advances in Cryptology—CRYPTO' 93*, Vol. 773 of *LNCS*, pp. 319–330, Berlin, Heidelberg, 1994. Springer.

[2] B. den Boer. More efficient match-making and satisfiability *The Five Card Trick*. In J.-J. Quisquater and J. Vandewalle eds., *Advances in Cryptology – EUROCRYPT' 89*, Vol. 434 of *LNCS*, pp. 208–217, Heidelberg, 1990. Springer.

[3] T. Mizuki and H. Shizuya. A formalization of card-based cryptographic protocols via abstract machine. *Int. J. Inf. Secur.*, 13(1):15–23, 2014.

[4] T. Mizuki and H. Sone. Six-card secure AND and four-card secure XOR. In X. Deng, J. E. Hopcroft, and J. Xue eds., *Frontiers in Algorithmics*, Vol. 5598 of *LNCS*, pp. 358–369, Berlin, Heidelberg, 2009. Springer.

[5] S. Ruangwises. Using five cards to encode each integer in Z/6Z. In P. Y. Ryan and C. Toma eds., *Innovative Security Solutions for Information Technology and Communications*, Vol. 13195 of *LNCS*, pp. 165–177, Cham, 2022. Springer.

[6] S. Ruangwises and T. Itoh. Securely computing the n-variable equality function with $2n$ cards. In J. Chen, Q. Feng, and J. Xu eds., *Theory and Applications of Models of Computation*, Vol. 12337 of *LNCS*, pp. 25–36, Cham, 2020. Springer.

[7] K. Shinagawa. *On the Construction of Easy to Perform Card-Based Protocols*. PhD thesis, Tokyo Institute of Technology, 2020.

[8] K. Shinagawa, T. Mizuki, J. Schuldt, K. Nuida, N. Kanayama, T. Nishide, G. Hanaoka, and E. Okamoto. Card-based protocols using regular polygon cards. *IEICE Trans. Fundam.*, E100.A(9):1900–1909, 2017.

© 2026 Information Processing Society of Japan